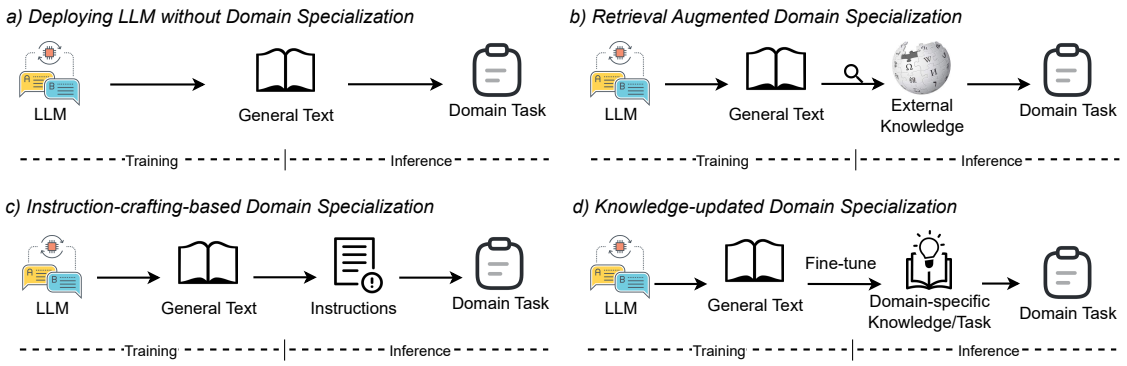

Fig. 2. This exposition discusses different approaches for tailoring LLMs to domain-specific tasks: (a) using an LLM trained on general corpora without modifications, (b) enhancing the LLM's performance through retrieving relevant external knowledge, (c) utilizing domain-specific and task-relevant instructions to improve LLM's capabilities, and (d) updating the LLM's internal knowledge with domain-specific text and tasks.

prompt to elicit domain knowledge better; and the white box *assumption* indicates we have full access to the LLM (e.g., LLaMA and its variants), including the parameter setting, training data, and the model architecture.

Other than the LLM accessibility-based taxonomy, one way to categorize LLM domain specialization methods is based on the training strategy used, such as fine-tuning an existing model with domain-specific data, training a model from scratch specifically for the domain, or employing a mixed training strategy. An additional taxonomy could be based on the intervention level: pre-training intervention involves modifying the pre-training process to encourage domain-specific knowledge, the fine-tuning intervention involves adaptations during the fine-tuning stage, and inference-time intervention involves modifying the model's behavior during the actual application to generate more domain-specific outputs. Furthermore, the taxonomy can be established based on the evaluation and feedback mechanism: fixed evaluation sets a constant benchmark, dynamic evaluation involves continuous performance assessment with changing benchmarks, and user feedback-based evaluation uses direct user input as a signal to specialize the model's responses.

In this survey, we categorize existing approaches based on the LLM's **accessibility** and provide an overview of each approach in Figure 2. To be more specific, *1) External augmentation (black box)* does not necessarily require access to the LLM's inner parameter space, making it the most accessible for users with limited resources (e.g., computational resources, domain-specific data). As shown in Figure 2 (b), by using external resources or tools, domain-specific knowledge is incorporated into the input prompt, generated output, or both, effectively adapting the LLM's performance without modifying its internal structure. *2) prompt crafting (grey box)* involves designing various types of prompts by accessing the gradient or loss values of LLMs, allowing for finer control over the model's behavior. *3) model fine-tuning (white box)* demands the most access and resources, as it involves updating the LLM's parameters to incorporate domain-specific knowledge directly into the model. (Figure 2 (d)).

Relations between approaches in different categories.

- **Different levels of specialization**: Each approach operates at a different level of specialization (i.e., black box, grey box, and white box). Augmenting with external knowledge provides a focused injection of domain-specific information while prompt engineering works at the input level, shaping the model's inference process. Fine-tuning modifies the LLM's internal parameters, leading to more profound changes in the model's behavior.

- **Trade-offs**: The approaches exhibit different trade-offs regarding computational cost, ease of implementation, and generalization. Augmenting with external information and crafting task-specific instructions are often less computationally expensive than knowledge updates of LLMs but may not yield the same level of performance improvement. Fine-tuning and neural adapters can provide more substantial performance gains but can be more challenging to implement and may suffer from reduced generalization capabilities if overfitting occurs.
- **Complementary nature**: The three approaches can be used independently or in combination to achieve better performance on domain-specific tasks. For instance, external knowledge can be integrated with a fine-tuned LLM to leverage both specialized knowledge and optimized parameters. Similarly, carefully designed prompts can be used alongside neural adapters to guide the model's output while taking advantage of the newly learned domain-specific knowledge.

Common Framework. Researchers can utilize these methods independently or in combination to achieve optimal performance on specific tasks while considering the unique requirements and constraints of each approach. In this paper, we provide a common framework underlying the black box, grey box, and white box methods for domain specialization of LLMs, which is a process consisting of four core stages: *Definition*, *Augmentation*, *Optimization*, and *Evaluation*.

(1) **Definition**: This is the first step where the specific domain, the objectives within that domain, and any constraints are clearly defined. Whether we fine-tune a model (white box), craft prompts (grey box), or augment inputs/outputs (black box), it requires a clear understanding of the domain we are specializing for. This also helps in identifying the specific data, knowledge, and resources relevant to the domain that could be used in the following steps.
(2) **Augmentation**: This stage involves the incorporation of domain-specific knowledge into the model, or its inputs/outputs. In a white box approach, this could involve fine-tuning the model with domain-specific data. For a grey box approach, it might involve using gradients or loss values to craft prompts that steer the model toward domain-specific responses. In a black box method, it could involve using external tools or resources to modify the input prompt or the generated output to make it more domain-specific.
(3) **Optimization**: Once the model or its inputs/outputs are augmented with domain knowledge, the next step is to optimize the model's performance to best fulfill the domain objectives. This can be done through methods like gradient descent for a white box approach, prompt engineering for a grey box approach, or post-processing and filtering of outputs for a black box approach.
(4) **Evaluation**: The final stage involves testing the specialized model's performance against predefined benchmarks, gathering feedback, and refining the model based on this feedback. This could involve running the model on a domain-specific test set or getting feedback from domain experts.

3 EXTERNAL AUGMENTATION FOR DOMAIN SPECIALIZATION

Retrieval augmentation aims to enhance LLMs by retrieving relevant information from external sources, without fine-tuning model parameters. There are two primary categories: (1) **Domain Knowledge Augmentation**, where LLMs are provided with domain-specific context from an external knowledge source, and (2) **Domain Tool Augmentation**, which integrates LLMs with external systems or tools, often via APIs. Domain Knowledge Augmentation supplements the model's responses with external information, while Domain Tool Augmentation expands the model's capabilities for tasks it couldn't perform otherwise. Domain Knowledge improves depth and accuracy within a specific field, while Domain Tools allow the model to perform tasks beyond its inherent abilities. This section discusses both approaches, their limitations, and their advantages.

3.1 Domain Knowledge Augmentation

Domain knowledge, in the broadest sense, is a comprehensive understanding of a specific field or subject area. It includes concepts, principles, facts, and patterns that are unique to a particular domain. The knowledge can be represented in various forms, including a set of documents, a domain-specific knowledge graph, or a neural network that contains parametric domain knowledge. Domain knowledge augmentation in LLM specification refers to the process of enriching an LLM's performance in specific domains by incorporating additional information from domain knowledge. Two categories of external knowledge typically can facilitate LLMs in their domain specialization: **explicit knowledge** refers to knowledge that is clearly defined, easily expressed, and structured in a manner that can be directly understood and utilized; and **implicit knowledge** refers to knowledge that is not directly stated or easily expressed, but is embedded within the data or the system, often in a latent, non-obvious form.

3.1.1 Utilizing Explicit Knowledge with LLM. A conventional method for customizing language models to domain-specific tasks is to retrieve domain-specific information from external context. When presented with an explicit knowledge source containing domain-specific information, it is crucial for LLMs to prioritize the context if the data source holds task-relevant details that contradict the model's memorized knowledge. This strategy ensures that model predictions are anchored in the context, allowing for the refinement or correction of specific model predictions without the need for frequent retraining.

Current techniques often employ a neural retriever to acquire task-relevant information from either a large corpus (e.g., Wikipedia) or a knowledge base (e.g., Wikidata) [10, 26, 44, 57, 74, 75, 83, 88, 143]. Specifically, given a task-specific query, early works [10, 74, 83, 143] designed neural retrievers to vectorize the query and all information in the external knowledge source to search for relevant information based on various similarity metrics (e.g., cosine similarity) in the latent space. The searched information can then be concatenated with the query for downstream tasks. With the prevalence of LLMs, researchers have been using LLMs to replace the neural network-based retriever [26, 57, 126], and one work [57] demonstrated that coupling a rather lightweight LLM (around 11 billion parameter size) with an external knowledge base can achieve similar performance when using a 540B LLM (i.e., PaLM). Furthermore, in order to enhance the transparency and explainability of the retrieval, He et al. [44] proposed to leverage LLMs to decompose the information retrieval process with detailed reasoning steps, and Lu et al. [88] explored to utilize LLMs to verify whether information obtained by a pre-trained neural-network-based retriever is relevant or not.

3.1.2 Utilizing Implicit Knowledge with LLM. Implicit domain knowledge in machine learning refers to latent, non-obvious information embedded within data or the system, often represented as vectorized knowledge or embeddings learned during pre-training. Such embeddings capture intricate data patterns, symbolizing domain knowledge in an abstract form. Previous research [36, 39, 99, 155] suggests the use of attention mechanisms to enable PLMs to retrieve task-related information from this implicit knowledge. These studies transform task-specific queries into latent embeddings, calculating attention scores between the query vector and each knowledge entry. A softmax function is used to generate a weight or probability distribution across all knowledge entries concerning the input query. The retrieved memory vector is then obtained via a weighted sum of the memory entries, using attention weights. This method enhances traditional neural networks with implicit knowledge, permitting the model to access relevant, current information during inference.

While LLMs can store a substantial amount of information in their parameters to generate high-quality responses, augmentation with implicit knowledge isn't always required. Unlike explicit knowledge, implicit knowledge requires